DECLARATION

I hereby declare that the project report submitted to the Department of Computer Science and Engineering, Rajshahi University, is my original work, completed independently. This report has not been submitted for any degree or qualification at any other institution. All planning, execution, and content are my own. Any external assistance received has been duly acknowledged, with gratitude expressed to those who contributed. This project, titled **VulnXposer: Website Vulnerability Scanner**, is the result of my research and development efforts.

Mohammad Omor Faruk
ID: 1937820111
Session: 2018-19
University of Rajshahi

CERTIFICATION

This is to certify that the project paper titled **VulnXposer: Website Vulnerability Scanner** has been reviewed and approved as meeting the requirements for the completion of B.Sc. Engineering in Computer Science and Engineering degree. More information provided below.

External Examiner

Name:
Designation:
Depertment of Computer Science and Engineering
University of Rajshahi, Rajshahi

Supervised By

Mohadeb Kumar
Lecturer, Dept. of CSE
TMSS Engineering College, Bogura.

Dil Afroz
Lecturer & Head,
Depertment of Computer Science and Engineering
TMSS Engineering College, Bogura

ACKNOWLEDGEMENT

I would like to begin by expressing my deepest gratitude to Almighty Allah (SWT) for His guidance and blessings, which have enabled me to complete this project successfully. I extend my heartfelt thanks to my supervisor, **Mohadeb Kumar**, Lecturer in the Computer Science and Engineering Department at TMSS Engineering College, for his outstanding leadership, insightful advice, and unwavering support. His thoughtful feedback and recommendations played a pivotal role in shaping this work.

I am also sincerely thankful to all my teachers in the CSE department for their invaluable contributions to my learning. To my parents, whose love and support have been my constant source of strength, I am eternally grateful. Lastly, I wish to thank my friends for their encouragement and assistance, especially during the more challenging moments of this journey.

ABSTRACT

VulnXposer is a web application designed to identify vulnerabilities in websites, aiming to help website owners protect their sites from potential cyber-attacks by detecting and reporting security flaws. The application employs various scanning techniques to evaluate websites against the OWASP Top 10 vulnerabilities, ensuring comprehensive coverage.

The development of VulnXposer involved a combination of modern technologies. The backend is built using **Node.js**, enabling efficient server-side processing, while the frontend is developed with **React.js**, providing an interactive user experience. Additionally, the application uses **Bash scripts** integrated within Node.js through the **exec()** function, along with **Python** and **Go**, to execute vulnerability scans. **Socket.io** is also utilized for real-time data updates, enhancing user engagement.

VulnXposer targets several customer segments, including website owners who wish to scan their own sites for vulnerabilities and security researchers who scan permitted websites. While a free plan offering essential features is planned, a static UI for various payment options has been added but is not yet implemented.

Looking ahead, there are plans to offer VulnXposer as a Software-as-a-Service (SaaS), allowing users to subscribe for additional features and capabilities.

Table of Contents

DECLARATION i
CERTIFICATION ii
ACKNOWLEDGEMENT iii
ABSTRACT iv
List of Figure vi
Chapter-1: Introduction 1
1.1 Background 1
1.2 Objectives 1
1.3 Scope of the Project 2
1.4 Limitations of the Project 3
Chapter 2: System Overview 4
2.1 Overview of VulnXposer 4
2.2 Target Audience 4
2.3 User Roles and Permissions 5
2.4 Registration and Login Mechanism 5
Chapter 3 : System Design 6
3.1 Architectural Design 6
3.2 Database Design 8
3.3 UI/UX Design 9
Chapter 4 : System Functionality 12
4.1 Key Features of VulnXposer 12
4.1.1 Port Scanning 12
4.1.2 Host Identification 12
4.1.3 DNS Analysis 12
4.1.4 CVE Detection 13
4.2 Real-Time Data Updates with Socket.io 13
4.3 Use Case Diagram 13
5.1 Frontend Technologies 15
5.2 Backend Technologies 15
Chapter 6 : System Implementation 16
6.1 Setting Up the Development Environment 16
6.2 Frontend Implementation 16

6.3 Backend Implementation 17
6.4 Integration of Scanning Features 17
Chapter 7 : Testing and Validation 19
7.1 Testing Strategies 19
7.2 Validation Methods 20
Chapter 8: Cost Analysis 21
8.1 Hardware Requirements 21
8.2 Software Requirements 21
8.3 Development Cost Breakdown 22
Chapter 9: Conclusion 23
9.1 Summary 23
9.2 Future Enhancements 24
References 25

List of Figure

Figure No	Figure Name	Page No
Figure 3.1	System Flow Chart of VulnXposer	7
Figure 3.2	User database of VulnXposer	8
Figure 3.3	Snapshot of user Dashboard	10
Figure 3.4	Snapshot of export report page	10
Figure 3.5	Snapshot of vulnerability report	11
Figure 4.1	Use case diagram of VulnXposer	14

Chapter-1: Introduction

1.1 Background

In the contemporary digital age, websites serve as the backbone for various services, including e-commerce, social networking, and information sharing. The rise of the internet has transformed how individuals and businesses interact, leading to an unprecedented reliance on online platforms. However, this reliance also comes with significant risks.

Cybersecurity threats are pervasive, with incidents such as data breaches, denial-of-service attacks, and malware injections becoming increasingly common. According to a report by Cybersecurity Ventures, global cybercrime costs are expected to reach $10.5 trillion annually by 2025, illustrating the urgent need for robust cybersecurity measures. Vulnerability scanning is a crucial practice for identifying security weaknesses in websites, enabling owners to proactively address these issues before they can be exploited by malicious actors.

VulnXposer is designed to be a solution to this growing concern. By providing comprehensive vulnerability assessments, the application empowers website owners to understand their security vulnerabilities and take appropriate action. Through consistent monitoring and timely reporting, VulnXposer contributes to a more secure online environment for users and businesses alike.

1.2 Objectives

The primary objectives of the VulnXposer project are as follows:

- **Identify Vulnerabilities:** The application aims to detect vulnerabilities in websites by scanning for common security flaws, such as SQL injection, cross-site scripting (XSS), and insecure server configurations. This proactive approach enables website owners to mitigate potential threats before they can be exploited.
- **User-Friendly Interface:** VulnXposer seeks to provide an intuitive and accessible user interface that simplifies the scanning process for users, regardless of their technical expertise. By employing clear navigation and straightforward instructions, users can easily initiate scans and interpret the results.
- **Comprehensive Reporting:** The application will generate detailed reports that outline vulnerabilities discovered during scans. Each report will include descriptions of the identified vulnerabilities, their potential impact, and recommended remediation steps, allowing users to prioritize their security efforts effectively.
- **Real-Time Updates:** By utilizing technologies like Socket.io, VulnXposer aims to provide real-time updates during the scanning process. Users will be able to see the status of their scans as they progress, enhancing the overall user experience and ensuring they are informed of any immediate findings.

- **Accessibility for All Users:** Offering a free plan ensures that essential scanning features are available to all website owners and security researchers. This inclusivity promotes widespread awareness of website vulnerabilities and encourages proactive security measures across diverse user groups.

1.3 Scope of the Project

The VulnXposer project is designed specifically for website vulnerability scanning and encompasses the following aspects:

- **Target Audience:** The primary users include website owners who want to secure their own sites from cyber threats and security researchers who have permission to scan third-party websites for vulnerabilities. By targeting these groups, VulnXposer aims to address the needs of both individual users and professional cybersecurity experts.
- **Vulnerability Detection:** The application will evaluate websites against the OWASP Top 10 vulnerabilities, which represent the most critical security risks to web applications. This framework serves as a standard for assessing and improving web application security.
- **Scan Types:** VulnXposer will incorporate various scanning techniques, such as port scanning, DNS analysis, and CVE (Common Vulnerabilities and Exposures) detection. These diverse scan types ensure a comprehensive assessment of the website's security posture, allowing users to address a wide range of vulnerabilities.
- **Future Plans:** The project will explore options for offering additional features and capabilities through a Software-as-a-Service (SaaS) model. This subscription-based approach will allow users to access premium features, such as advanced reporting, scheduled scans, and personalized support, fostering a sustainable business model.

1.4 Limitations of the Project

Despite VulnXposer's strengths, certain limitations could affect its usability and effectiveness:

1. **Permission Requirements**:
   VulnXposer operates in line with ethical hacking principles, meaning users must obtain explicit permission from website owners before running any scans. This requirement limits its use in certain situations, particularly for security researchers or penetration testers who may want to test public websites without formal authorization. This restriction ensures that VulnXposer complies with legal frameworks and protects the rights of website owners, but it may reduce the tool's usability in broader security research.
2. **False Positives**:
   Like many automated security tools, VulnXposer may occasionally report vulnerabilities that do not exist, known as false positives. These can create confusion for users, as they may save time investigating issues that are not real threats. While the tool is designed to minimize such errors, users should still manually verify the reported vulnerabilities, especially when critical decisions are involved. This additional verification process can be time-consuming and may require a level of technical expertise beyond basic usage.
3. **Dependency on User Input**:
   The accuracy and depth of VulnXposer's scans are heavily dependent on the quality of the information provided by the user. For example, users must input correct URLs, relevant parameters, and any authentication credentials for sites that require login access. Inaccurate or incomplete data could lead to partial or faulty scan results, reducing the tool's effectiveness. Ensuring accurate input is essential for obtaining comprehensive security assessments.
4. **Initial Feature Set**:
   Although VulnXposer has a solid foundation of features, it lacks certain advanced capabilities that may be required for more sophisticated vulnerability analysis. Features such as detailed analytics, behavioral pattern recognition, or machine learning-driven insights are not included in the initial version. These enhancements are planned for future releases, which will broaden the tool's scope and make it more competitive in the vulnerability scanning market. For now, the focus remains on core functionality, like OWASP Top 10 vulnerability detection and essential security features.
5. **Real-Time Data Processing**:
   VulnXposer provides real-time data updates during the scanning process via socket.io integration. However, this feature may sometimes face delays or interruptions depending on the network conditions and server performance. Users relying on real-time updates may experience slower responses, especially when scanning large or complex websites.

Chapter 2: System Overview

2.1 Overview of VulnXposer

VulnXposer is a comprehensive web vulnerability scanning application designed to help website owners and security professionals identify potential security flaws in websites. Here are some key features and components:

- **Purpose**: The primary goal is to assist users in securing their websites against cyber threats by detecting vulnerabilities.
- **Scanning Techniques**: Utilizes various scanning methodologies to identify issues such as:
    - Open ports
    - Outdated software
    - Common security risks as outlined in the OWASP Top 10
- **User Interface**: Combines a user-friendly interface with advanced back-end processing for a seamless user experience, accommodating both technical and non-technical users.
- **Real-Time Updates**: Implements Socket.IO for real-time data updates during scans, ensuring users are informed instantly about scanning progress.
- **Reporting**: Generates detailed reports that include actionable insights, allowing users to take prompt actions to secure their websites.

2.2 Target Audience

The target audience for VulnXposer includes:

- **Website Owners**: Individuals or organizations that want to secure their websites by identifying vulnerabilities.
- **Security Researchers**: Professionals with authorized access who assess permitted websites for flaws.
- **Small and Medium-Sized Businesses (SMBs)**: Organizations seeking an accessible and affordable solution for regular security assessments.
- **IT Professionals**: Technicians looking for an easy-to-use tool to enhance their cybersecurity measures.
- **Free Tier Users**: Users who want to explore the basic features before committing to a paid plan.

2.3 User Roles and Permissions

VulnXposer features role-based access control to enhance security and usability. The system supports two primary user roles:

- **Admin Users**:
  - **Full Access**: Complete control over all system functionalities.
  - **User Management**:
    - Can edit, and delete user accounts.
    - Can change user roles (Admin or Regular User).
    - Can modify usernames and access levels.
  - **Configuration**: Responsible for system configuration and settings.
- **Regular Users**:
  - **Scanning Access**: Ability to initiate scans and view reports.
  - **Profile Management**: Can manage their own profiles, including password changes.
  - **Limited Access**: Restricted from accessing admin functionalities.

This structure ensures that permissions are appropriately managed, minimizing security risks.

2.4 Registration and Login Mechanism

The registration and login mechanism in VulnXposer is designed for both security and user convenience:

- **User Registration**:
  - Users provide essential information such as:
    - Name
    - Password
  - **Access Token**: Upon successful registration, users receive an access token for secure session management.
- **Local Storage**:
  - User preferences and session data are saved in local storage to enhance the user experience.
- **Secure Login**:
  - Users can log in with their credentials.
  - Passwords are stored securely using hashing and encryption methods.
- **Dashboard Access**:
  - After logging in, users are directed to their personalized dashboard, displaying:
    - Recent scan results
    - Available scanning options
    - Account settings

Chapter 3 : System Design

3.1 Architectural Design

The architectural design of VulnXposer is structured to provide a scalable, efficient, and secure environment for vulnerability scanning. Here are the key components:

- **Client-Server Architecture**:
  - **Frontend**: Developed using React.js, it ensures a responsive and interactive user experience. The front end communicates with the backend via RESTful APIs.
  - **Backend**: Built on Node.js, which allows for efficient handling of multiple requests and real-time data processing.
- **Microservices Approach**:
  - Each functionality (e.g., scanning, reporting) is encapsulated within individual microservices, facilitating easier updates and maintenance.
- **Real-Time Communication**:
  - Utilizes Socket.IO to enable real-time updates during vulnerability scans, providing immediate feedback to users about the scan progress.
- **Security Measures**:
  - Implements SSL/TLS for secure data transmission.
  - User authentication and authorization are managed using JWT (JSON Web Tokens) to ensure that only authorized users access sensitive information.
- **Scalability**:
  - The architecture is designed to scale horizontally, allowing the addition of more instances of microservices to handle increased load effectively.

System Flow chart of VulnXposer

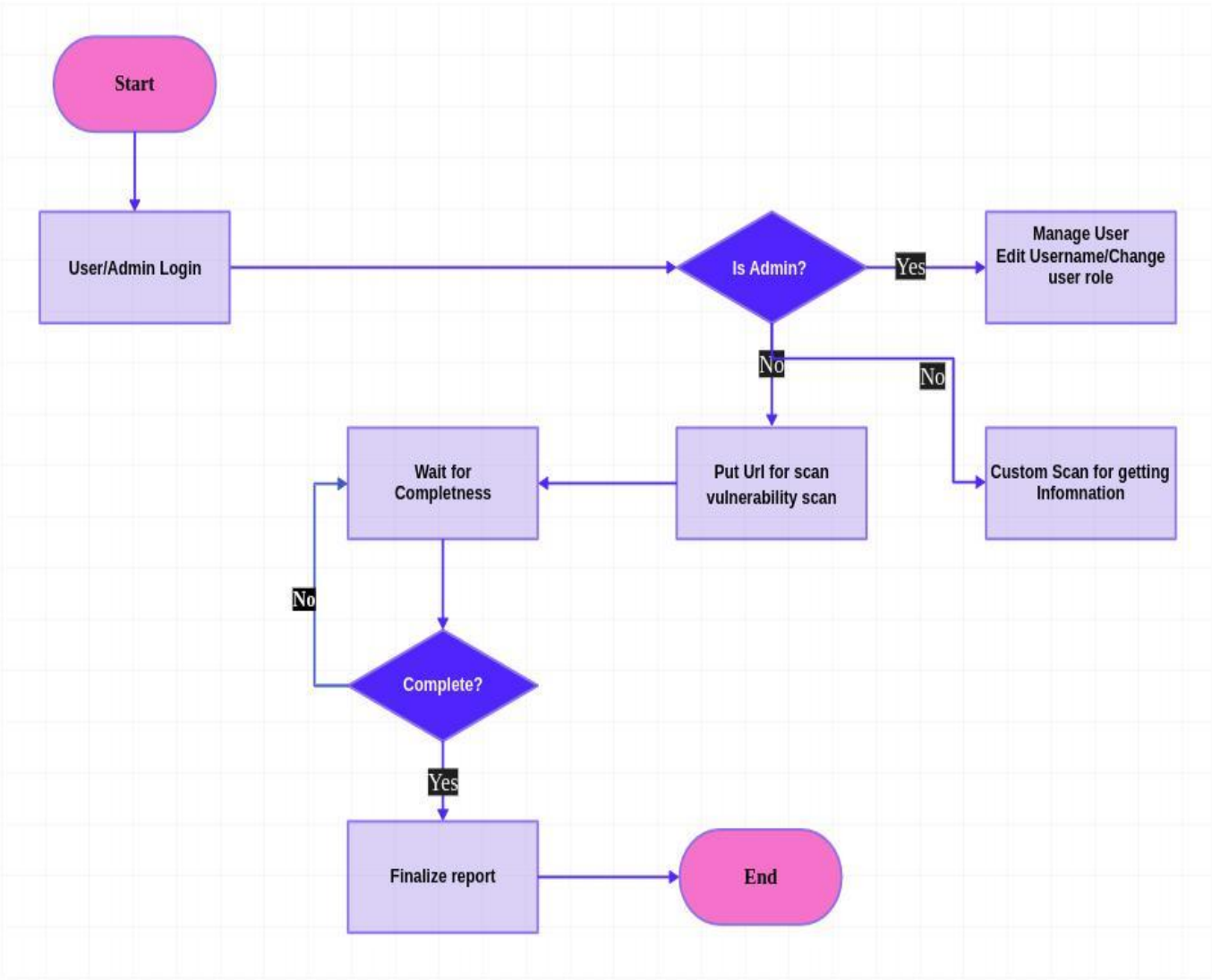

Figure 3.1: System Flow chart of VulnXposer

3.2 Database Design

The database design for VulnXposer is crucial for storing and managing user information. But as it is not saving scan results for now. There is only a single table for user management.

- **Database Type**:
  - MySQL database.
- **Main Collections**:
  - **Users Collection**:
    - Fields include:
      - id (char)
      - username (varchar)
      - password (varchar)
      - role (enum ‘Admin/User’)
- **Data Relationships**:
  - Users can scan the website after login. For now, user can not save their scans.

User Table

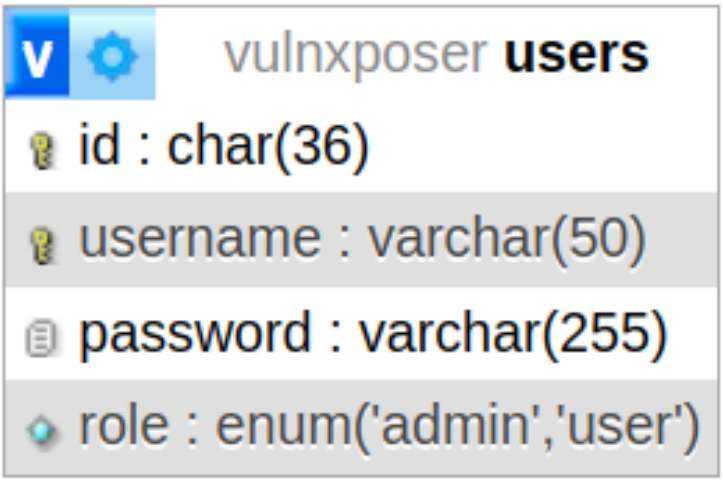

Figure 3.2: User database of VulnXposer

3.3 UI/UX Design

The UI/UX design of VulnXposer focuses on providing an intuitive and user-friendly experience for both technical and non-technical users. Here's an overview:

- **Landing Page**:
  - An engaging layout that introduces VulnXposer's purpose and key features.
  - Clear call-to-action buttons for registration and login.
- **Dashboard**:
  - A central hub for users to manage scans, view results, and access settings.
  - **Sidebar Navigation**: Easy access to different sections, including:
    - All Targets
    - OSINT Tools For Individual Scan
    - Vulnerability Report Export
    - User Profile Management
- **Scan Results Page**:
  - Presents scan results in an easy-to-read format, including:
    - Summary of findings
    - Detailed reports for each detected vulnerability
    - Options to export results as PDF or HTML
- **User Management Interface** (Admin only):
  - Simple forms for adding, editing, or deleting users.
  - A clear indication of user roles and permissions.
- **Color Scheme and Typography**:
  - A modern color palette that enhances readability and user engagement.
  - Consistent typography to ensure information is presented clearly.
- **Responsive Design**:
  - Ensures the application is accessible on various devices, including desktops, tablets, and smartphones.
- **Future Enhancements**:
  - Plans for incorporating user feedback mechanisms to continuously improve the UI/UX based on user experience.

Snapshot of VulnXposer user dashboard (For scanning websites)

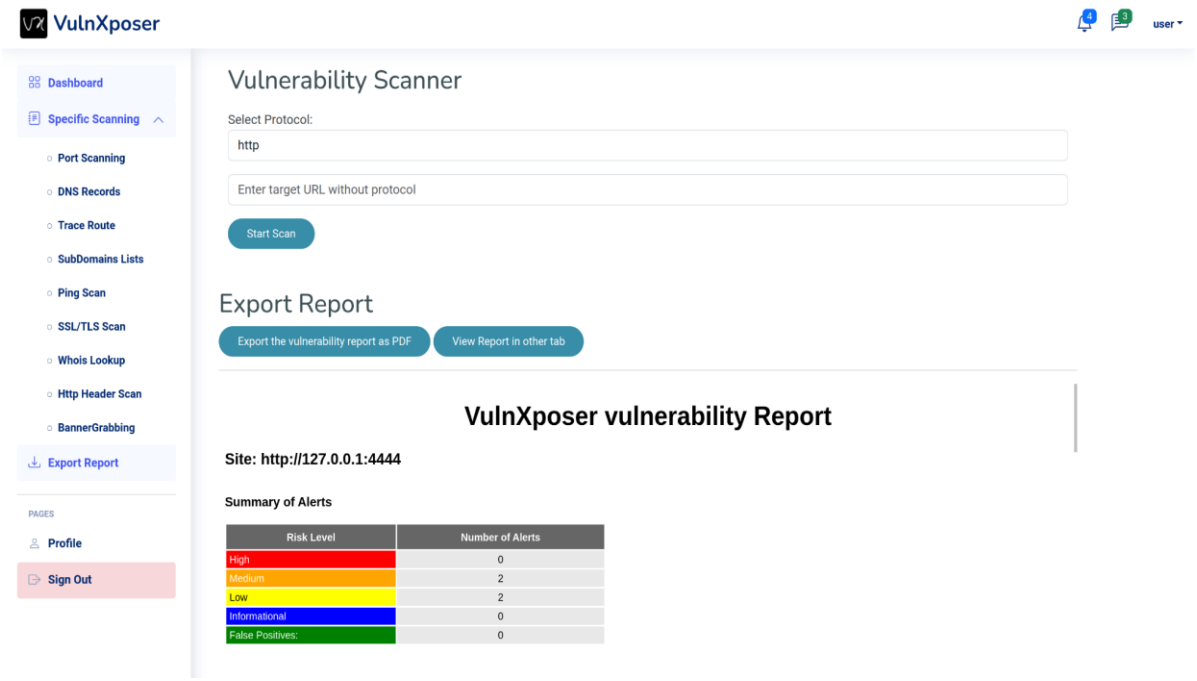

Figure 3.3: Snapshot of user dashboard

Snapshot of VulnXposer export report page

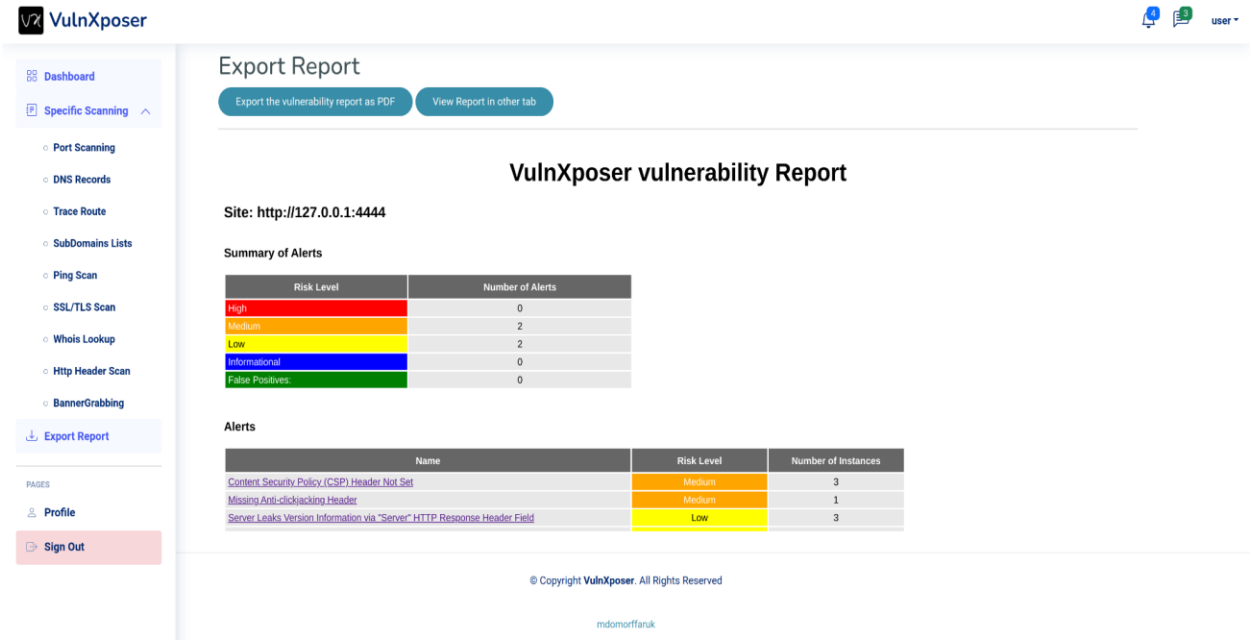

Figure 3.4: Snapshot of export report page

Snapshot of VulnXposer view report

VulnXposer vulnerability Report

Site: http://127.0.0.1:4444

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	2
Low	2
Informational	0
False Positives:	0

Alerts

Name	Risk Level	Number of Instances
Content Security Policy (CSP) Header Not Set	Medium	2
Missing Anti-clickjacking Header	Medium	1
Server Leaks Version Information via "Server" HTTP Response Header Field	Low	3
X-Content-Type-Options Header Missing	Low	1

Alert Detail

Medium	Content Security Policy (CSP) Header Not Set
Description	Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.
URL	http://127.0.0.1:4444
Method	GET
Parameter	
Attack	
Evidence	
Other Info	
URL	http://127.0.0.1:4444/sitemap.xml
Method	GET

Figure 3.5: Snapshot of vulnerability report

Chapter 4 : System Functionality

4.1 Key Features of VulnXposer

VulnXposer is designed with several core features that assist users in identifying vulnerabilities in their websites. Each of these features plays a vital role in providing a comprehensive vulnerability assessment.

4.1.1 Port Scanning

- **Purpose**: Port scanning identifies open or vulnerable ports on the target system, helping users understand which entry points could be exploited by attackers.
- **Working**: The system sends packets to the target system's ports and listens for responses to determine which ports are open or closed.
- **Example**: A scan may reveal that port 80 (HTTP) is open, signaling that the web server is accessible, or it may flag unusual ports as potentially dangerous.

4.1.2 Host Identification

- **Purpose**: Host identification ensures that the system recognizes and maps the devices or hosts connected to the network.
- **Working**: This feature checks the status and configuration of target hosts and generates a list of identified devices along with their IP addresses.
- **Example**: It can detect the web server's IP address and check for potential vulnerabilities related to the hosting environment.

4.1.3 DNS Analysis

- **Purpose**: DNS analysis investigates the domain name system of the target website to uncover misconfigurations or potential security weaknesses.
- **Working**: It performs lookups, checks DNS records (e.g., A, MX, CNAME), and identifies any potential vulnerabilities like DNS spoofing or cache poisoning.
- **Example**: Anomalies such as outdated or incorrect DNS records that may lead to subdomain takeover or phishing attacks.

4.1.4 CVE Detection

- **Purpose**: CVE (Common Vulnerabilities and Exposures) detection identifies known vulnerabilities based on the CVE database.
- **Working**: The scanner cross-references software versions and configurations against a global database of vulnerabilities.
- **Example**: A vulnerable version of Apache might be detected, with a reference to its corresponding CVE entry, warning users to update.

4.2 Real-Time Data Updates with Socket.io

- **Purpose**: Real-time data updates provide immediate feedback to users during the scanning process.
- **Implementation**: Socket.IO is used to enable bi-directional communication between the server and the client.
    - When the scanner is running, progress is streamed to the user dashboard in real time.
    - Users can view the scanning results dynamically, reducing the need to refresh the page or check manually.
- **Key Advantages**:
    - Real-time updates enhance user engagement and allow quick actions if a critical vulnerability is detected during the scan.
    - Improves system efficiency by eliminating the need for frequent polling for updates.

4.3 Use Case Diagram

The Use Case Diagram illustrates the interaction between users and the core functionalities of VulnXposer.

Actors:

1. **Website Owner**: The primary user who scans their own website.
2. **Security Researcher**: A professional who scans permitted websites for vulnerabilities.
3. **Admin**: Manages user accounts, roles, and permissions.

Key Use Cases:

- **Register/Login**: Users create accounts and log in to access the scanning tools.
- **Manual Scan**: Users can input the target website and initiate a scan.
- **Automatic scan and view results**: After the scan, users can view detailed reports, export them, and manage findings.
- **User Management (Admin)**: Admin users can add, edit, or delete users and modify roles.

Use Case Diagram

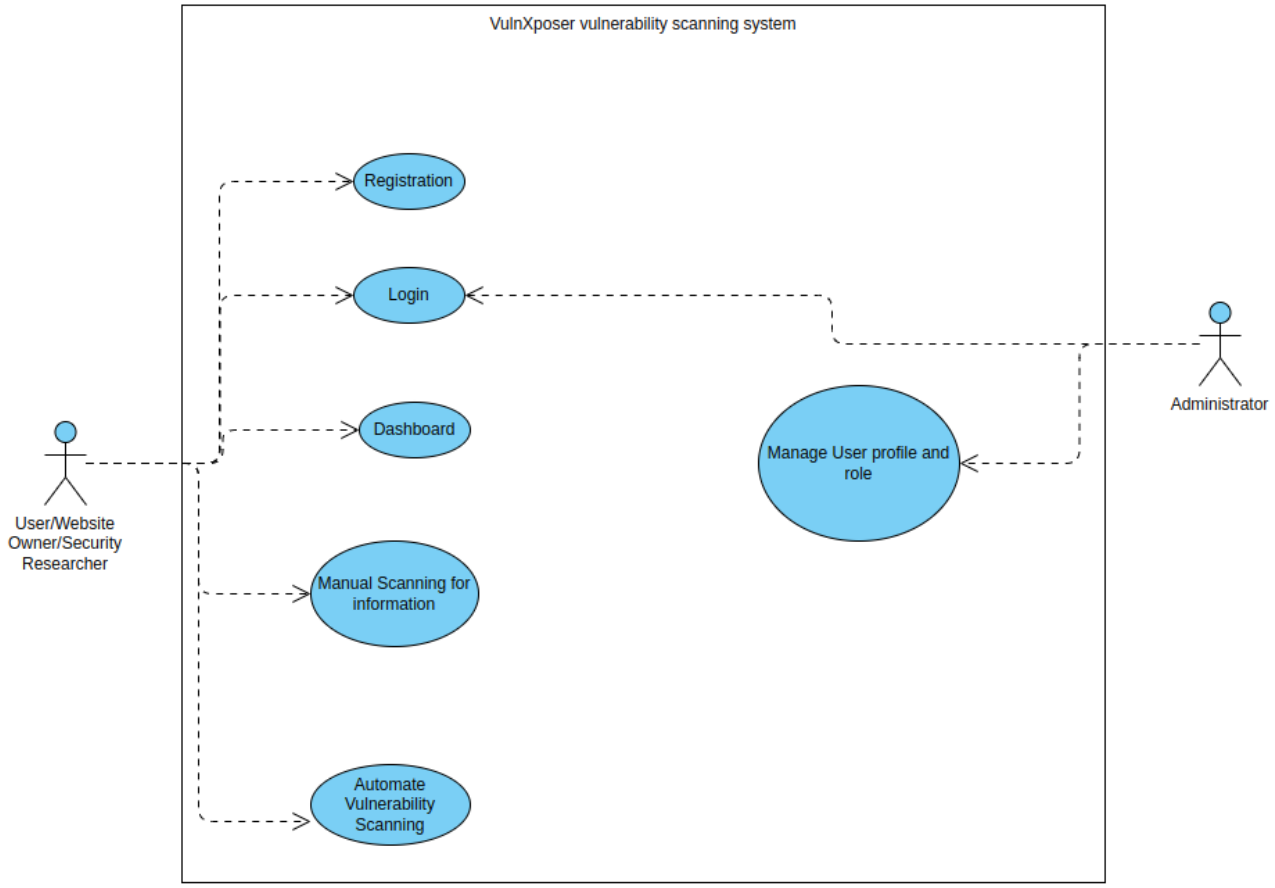

Figure 4.1: Use Case Diagram of VulnXposer

Chapter 5 : Technologies Used

5.1 Frontend Technologies

5.1.1 React.js

- **Component-Based Architecture:** React.js allows the VulnXposer frontend to be built in a modular fashion, using reusable components for different parts of the user interface like forms, dashboards, and navigation bars.
- **State Management:** React's state management makes it easy to handle dynamic data like scan results and user input, providing a responsive experience.
- **Virtual DOM:** React leverages a virtual DOM for faster updates, making the UI responsive and reducing unnecessary re-rendering, crucial for real-time updates during vulnerability scans.

5.1.2 Bootstrap

- **Responsive Design:** Bootstrap ensures that VulnXposer's UI is accessible and well-displayed across devices, from desktops to smartphones.
- **Pre-built Components**: The inclusion of Bootstrap’s pre-built components, such as buttons, forms, and modals, speeds up development while maintaining a consistent look.
- **Custom Themes:** Bootstrap allows for customization of the color palette, ensuring VulnXposer has a professional yet approachable design.

5.2 Backend Technologies

5.2.1 Node.js

- **Efficient Non-blocking I/O:** Node.js handles multiple requests simultaneously, enabling fast processing of vulnerability scans and report generation without slowing down the system.
- **Express.js Framework:** The backend uses Express.js to handle API requests, route handling, and middleware, streamlining the development process.
- **RESTful API:** Node.js allows the creation of RESTful APIs that serve as a bridge between the frontend and backend, facilitating data communication.

5.2.2 Bash Scripts

- **System-Level Operations:** Bash scripts execute key system-level scanning tasks, such as invoking Nmap for port scanning or running DNS analysis commands.
- **Node.js Integration:** These scripts are integrated within the Node.js environment using the exec() function, ensuring smooth execution of external commands and scripts from within the application.

5.2.3 Python and Go

- **Python:** Python is used for running vulnerability scanners like OWASP ZAP, which provides detailed security reports. Python's extensive libraries for networking and security make it ideal for this task.
- **Go:** The Go programming language is used to enhance performance in certain tasks that require concurrency and fast execution, such as OSINT gathering and multi-threaded scanning operations.

Chapter 6 : System Implementation

6.1 Setting Up the Development Environment

- **Development Tools**:
  - **Node.js** (Backend): Install the latest version of Node.js, which includes npm (Node Package Manager) for managing dependencies.
  - **React.js** (Frontend): Use the create-react-app command to quickly set up a React.js project.
  - **Database**: Install and configure MySQL for storing user data, scan results, and system configurations.
  - **Version Control**: Set up Git for tracking code changes. A GitHub repository is used to host the project codebase.
  - **IDE**: Use Visual Studio Code as the integrated development environment for writing, testing, and debugging the code.
  - **Command Line Tools**: Install required Bash, Python, and Go tools to run the scanning features and system-level commands.

6.2 Frontend Implementation

- **React.js Setup**:
  - The frontend uses React.js, and the project is initialized using the create-react-app template.
  - The components are structured to ensure modularity, with separate components for the dashboard, user profile, scanning reports, and settings.
- **State Management**:

    - React's useState and useEffect hooks are used to manage the state of the application, especially for dynamic updates like scan results and user actions.
    - LocalStorage is used to store user tokens, ensuring that the authentication persists even after page reloads.
- **Responsive Design**:
    - Bootstrap is used to create a responsive layout that works well on different devices. Media queries are also applied to fine-tune the layout.
- **User Authentication**:
    - A token-based authentication system is implemented, where the user receives a JWT token upon successful login. The token is stored in LocalStorage and sent with each API request for validation.

6.3 Backend Implementation

- **Node.js & Express**:
    - The backend is set up using Express.js to create a RESTful API that handles authentication, scanning requests, and user management.
    - **JWT Authentication**: Upon user login, a JSON Web Token (JWT) is generated and returned to the client. The backend verifies the token to restrict access to certain routes.
- **Database Integration**:
    - MySQL is used as the database for storing user data and system configurations.
    - **User Table**: The users table includes fields like username, password, and role. Passwords are hashed using the bcrypt library for security.
- **Socket.io for Real-Time Updates**:
    - The backend uses Socket.IO to send real-time data updates to the client during scanning operations. This ensures that the user sees the progress and results of their scans immediately.

6.4 Integration of Scanning Features

Port Scanning:

- VulnXposer integrates Nmap for port scanning, executed via Bash scripts using Node.js's exec() function. The tool scans open ports on the target website to identify services running on specific ports. Results are displayed to the user in real-time, including the status of common ports (e.g., HTTP, HTTPS, FTP).

DNS Analysis:

- DNS analysis is performed using Bash scripts wrapped in Python. The scan retrieves DNS records (A, MX, NS, TXT, etc.), nameserver information, and other essential details

about the domain's DNS configuration. It helps identify misconfigurations or possible attack vectors related to DNS settings.

Traceroute:

- The traceroute scan tracks the route packets take from VulnXposer to the target server. Bash scripts are used to execute the traceroute command, which identifies each hop along the network path. The information helps users identify latency issues and detect potential network vulnerabilities.

Subdomains:

- VulnXposer includes a subdomain enumeration scan that identifies potential subdomains linked to the target website. The tool uses Bash scripts and online services to discover these subdomains, revealing additional attack surfaces that could be exploited by attackers.

Ping:

- The ping feature checks the network connectivity between VulnXposer and the target website by sending ICMP packets. Bash scripts execute the ping command to determine the latency and packet loss, helping assess the network's stability and responsiveness.

SSL/TLS:

- This scan assesses the security of the SSL/TLS certificates in use on the target website. The SSL/TLS scan checks for issues like weak cipher suites, expired certificates, and the overall configuration of the SSL/TLS encryption. The tool ensures that the website meets modern security standards.

WHOIS Lookup:

- The WHOIS lookup feature retrieves the domain's registration information, including ownership, registrar, and expiration date. Bash scripts gather and display this information, providing insight into the domain's administrative data.

HTTP Header:

- VulnXposer checks the HTTP headers returned by the target website. This scan provides insights into the server's configuration, including security headers (e.g., Content-Security-Policy, X-Frame-Options). It helps identify misconfigurations that could leave the site vulnerable to attacks.

Banner Grabbing:

- Banner grabbing is used to determine the software and version running on a specific service by capturing the banner returned by the service. This information is useful for vulnerability assessment by matching software versions with known exploits.

Automatic Scan (OWASP ZAP Integration):

- VulnXposer integrates with OWASP ZAP for an automatic vulnerability scan. OWASP ZAP, an open-source security tool, automatically checks the target website for vulnerabilities, particularly the OWASP Top 10 risks. Once the scan is complete, the tool generates a detailed report, summarizing the identified issues and their severity levels.

Chapter 7 : Testing and Validation

7.1 Testing Strategies

To ensure the reliability, security, and functionality of VulnXposer, a comprehensive testing strategy was implemented, focusing on multiple aspects of the system:

- **Unit Testing**:
  - Individual components of the application, such as the scanning modules (port, DNS, HTTP headers, etc.), will be tested in isolation to verify that each performs as expected.
  - Node.js unit testing frameworks (such as Mocha or Jest) will be employed to automate these tests and catch any issues early in the development process.
- **Integration Testing**:
  - Tests were conducted to ensure that different parts of the system—such as the frontend (React.js) and backend (Node.js)—work together seamlessly.
  - Specifically, tests ensured that the API calls between the frontend and backend correctly fetch and display scan results in real-time.
- **Security Testing**:
  - Since VulnXposer is a vulnerability scanner, the security of the application itself was a priority. The system was tested for potential security flaws, including SQL injection, cross-site scripting (XSS), and cross-site request forgery (CSRF).
  - The platform was also subjected to penetration tests using OWASP ZAP to detect any vulnerabilities in its own code.
- **Performance Testing**:
  - Performance tests will be conducted to assess how the system performs under load, especially during simultaneous vulnerability scans.
  - Tools like Apache JMeter were used to simulate multiple users scanning different targets simultaneously, testing the scalability of the architecture.
- **User Acceptance Testing (UAT)**:
  - Before the final deployment, the application will be tested by a group of users, including website owners and security researchers, to ensure the system met their expectations and performed the intended tasks without issues.

7.2 Validation Methods

Validation is a crucial part of the testing phase to ensure that the system fulfills its requirements and meets user needs:

- **Functionality Validation**:
  - The features of VulnXposer, including manual and automatic scans, were validated against the initial requirements. Each feature was checked for completeness and correctness.
- **Real-Time Data Validation**:
  - The real-time updates provided during scans were validated by comparing the actual progress with the visual feedback provided to the user via the Socket.IO interface.
- **User Input Validation**:
  - All input fields, including URLs and credentials for authenticated scans, were validated to ensure they accept correct data formats and reject invalid inputs.
- **Scan Accuracy Validation**:
  - The results from VulnXposer scans were cross-checked with other industry-standard tools (like Nmap and OWASP ZAP) to validate the accuracy of the results, minimizing false positives and negatives.
  - Special focus was given to CVE detection, ensuring the vulnerabilities reported are accurate and up-to-date based on the latest CVE database.
- **Cross-Browser and Cross-Device Validation**:
  - The web application was validated on different browsers (Chrome, Firefox, Safari) and devices (desktop, tablet, and mobile) to ensure consistent user experience across all platforms.

Chapter 8: Cost Analysis

8.1 Hardware Requirements

For the development and deployment of VulnXposer, specific hardware is required to ensure smooth operation and processing efficiency.

- **Development Environment**:
  - **Processor**: Intel Core i5 or higher / AMD Ryzen 5 or higher
  - **RAM**: 8GB (16GB recommended for faster builds and testing)
  - **Storage**: 256GB SSD (minimum)
  - **Graphics**: Integrated or dedicated graphics (optional)
- **Server Environment** (for hosting the application):
  - **Processor**: Dual-core or higher
  - **RAM**: 8GB or more (16GB+ recommended for larger traffic)
  - **Storage**: 512GB SSD or more
  - **Network**: Minimum 100 Mbps for handling multiple scan requests

8.2 Software Requirements

The software tools required for VulnXposer include free and open-source tools as well as some optional paid tools:

- **Development Tools**:
  - **Frontend Development**: React.js, HTML, CSS, JavaScript, Bootstrap
  - **Backend Development**: Node.js, Express.js
  - **Scripting**: Python, Go, Bash Scripts
  - **Database**: MySQL (open-source)
  - **Version Control**: Git, GitHub
  - **Package Manager**: npm for Node.js dependencies
  - **IDE**: Visual Studio Code (free) or JetBrains WebStorm (paid)
  - **Testing Tools**: Jest (frontend), Mocha (backend), Apache JMeter (performance)
- **Server and Hosting**:
  - **Operating System**: Ubuntu 20.04 LTS or higher
  - **Web Server**: Nginx or Apache
  - **SSL/TLS**: OpenSSL
  - **Domain**: Domain registration cost (varies)
  - **Hosting Platform**: AWS, DigitalOcean, or other cloud services (costs depend on usage)

8.3 Development Cost Breakdown

Here is an estimate of the development costs in BDT:

- **Hardware Costs**:
  - **Development Machines**: BDT 80,000 - 120,000 per machine
  - **Server Setup** (for self-hosting): BDT 5,000 - 10,000 per month (cloud hosting like AWS, DigitalOcean)
- **Software Costs**:
  - Most tools (React.js, Node.js, MySQL) are open-source.
  - Optional paid software (IDE like WebStorm): BDT 5,000 - 6,500 annually
  - Hosting service: BDT 1,000 - 10,000 per month (depending on traffic and storage)
- **Personnel Costs**:
  - **Frontend Developer**: BDT 3,000 - 5,000 per hour
  - **Backend Developer**: BDT 4,000 - 6,000 per hour
  - **Security Expert**: BDT 5,000 - 8,000 per hour (for security validation)
  - **UI/UX Designer**: BDT 2,500 - 4,500 per hour
  - **Project Management**: BDT 2,000 - 4,000 per hour
- **Total Estimated Development Cost**:
  - **Personnel**: BDT 20,00,000 - 40,00,000 (depending on the team size and duration of 3-6 months)
  - **Hardware and Software**: BDT 1,50,000 - 2,50,000
  - **Hosting and Domain**: BDT 10,000 - 1,00,000 annually

Chapter 9: Conclusion

9.1 Summary

In this project, we developed **VulnXposer**, a comprehensive website vulnerability scanner aimed at identifying and reporting security flaws that could jeopardize web applications. The tool integrates both manual and automated scanning features, leveraging the power of **OWASP ZAP** for effective vulnerability detection. Key functionalities include port scanning, DNS analysis, SSL/TLS testing, and HTTP header analysis, all designed to enhance the security posture of websites.

Throughout the development process, we faced various challenges, including selecting the appropriate technologies and ensuring the accuracy of vulnerability detection. However, by focusing on user feedback and iterating on design, we successfully created a tool that is both user-friendly and effective in improving website security. The impact of **VulnXposer** is significant, as it empowers developers and organizations to proactively address vulnerabilities, ultimately contributing to a safer online environment.

Looking ahead, we plan to develop **VulnXposer** as a **Software as a Service (SaaS)** platform. This transition will enable users to save their scan data, manage multiple projects, and access their results from anywhere. Additional functionalities will include:

- **User Accounts and Profiles:** Allowing users to create accounts, log in, and securely save their scan history for easy access.
- **Collaboration Tools:** Features enabling teams to collaborate on scan results, share reports, and assign tasks for vulnerability remediation.
- **Automated Scans:** Scheduling automated scans to run at regular intervals, providing continuous monitoring and updates on website security.
- **Integration with CI/CD Pipelines:** Seamlessly integrating with development workflows to catch vulnerabilities early in the development cycle.
- **Customizable Notifications:** Setting up alerts for new vulnerabilities or changes detected in previously scanned websites.

9.2 Future Enhancements

As we pursue the SaaS model and the additional functionalities outlined, several enhancements could further improve **VulnXposer**. Key areas for development include:

- **Integration of Machine Learning:** Implementing machine learning algorithms could enhance vulnerability detection accuracy and provide predictive insights into potential security threats.
- **Expanded Reporting Features:** Enhancing the reporting capabilities to include customizable templates and visual analytics would allow users to better interpret and communicate findings.
- **Real-Time Monitoring:** Developing a real-time monitoring feature to continuously assess websites for emerging vulnerabilities and changes in their security posture.
- **User Community and Contributions:** Establishing an open-source community around **VulnXposer** to encourage contributions from developers and security experts, ensuring the tool remains up-to-date with the latest security trends and practices.
- **Enhanced User Interface:** Iterating on the UI/UX design to improve user experience, particularly for less technical users, would make the tool more accessible.

By pursuing these enhancements and transitioning to a SaaS model, **VulnXposer** can remain at the forefront of web security solutions, adapting to the evolving landscape of cybersecurity threats.

References

[1] Nmap. (2021). Nmap Network Scanning. [Online]. Available: https://nmap.org/book/

[2] Internet Systems Consortium. (2021). BIND 9: DIG Command. [Online]. Available: https://linux.die.net/man/1/dig

[3] The Linux Foundation. (2021). Traceroute: Linux Networking Tool. [Online]. Available: https://www.linuxjournal.com/content/traceroute-linux-networking-tool

[4] Subfinder. (2021). Subfinder: A Subdomain Discovery Tool. [Online]. Available: https://github.com/projectdiscovery/subfinder

[5] OpenSSL. (2021). OpenSSL: The Open Source Toolkit for SSL/TLS. [Online]. Available: https://www.openssl.org

[6] ICANN. (2021). WHOIS: A Protocol for Querying Internet Domain Name Registries. [Online]. Available: https://linux.die.net/man/1/whois

[7] Curl. (2021). Curl: Command Line Tool and Library for Transferring Data with URLs. [Online]. Available: https://curl.se

[8] Netcat. (2021). Netcat: The TCP/IP Swiss Army Knife. [Online]. Available: https://nc110.sourceforge.net

[9] OWASP. (2021). ZAP: Zed Attack Proxy. [Online]. Available: https://www.zaproxy.org

[10] Ristic, I. (2013). SSL Labs: Analyzing SSL Configuration. [Online]. Available: https://www.ssllabs.com/ssltest/